

COMP1702 – Big Data

2025-26 - Term 2

Coursework



Dishan Bhandari

001483674

MSc. Data Science

Task A:

Big Data refers to massive, complex datasets that traditional management systems cannot handle. Its concept is characterized by its four “Vs”: Volume, Velocity, Variety and Veracity (IBM, 2024).

1. **Volume** refers to the sheer scale and magnitude of data generated and stored. Over 79 zettabytes of data have been generated worldwide, and 2.5 quintillion bytes of data are created every 24-hours (Omoyiola, 2022). As of Oct 2025, the total number of active social media user is reported to be 5.66 billion, who are expected to drive data growth (Slotta, 2025).
2. **Velocity** means the speed at which data is generated, collected and processed. Humans generate data at rapid rates. For example, every minute, global users (*Data Never Sleeps 11.0*, 2023):
 - Conduct 6.3 million Google searches
 - Send 2341 million emails
 - Stream 43 years’ worth of content
3. **Variety** refers to the many different formats that big data can take from structured to unstructured data, and semi-structured data. In retail, Walmart leverages its social genome combining public web data, social media data and proprietary data like purchasing data to monitor customer sentiments and preferences that traditional sales data cannot capture (ProjectPro, 2024).
4. **Veracity** denotes the reliability, accuracy, and trustworthiness of data. A notable example is the “Google Flu Trends” project which was initially aimed to predict flu outbreaks but ultimately suffered from significant inaccuracies due to issues with data quality and search query dynamics (Lazer *et al.*, 2014).

Table of References

ProjectPro. (2024) *How Big Data Analysis helped increase Walmart's Sales turnover?*.

Available at: <https://www.projectpro.io/article/how-big-data-analysis-helped-increase-walmarts-sales-turnover/109> (Accessed: 22 March 2026).

Data Never Sleeps 11.0 (2023). Available at: <https://www.domo.com/learn/infographic/data-never-sleeps-11> Accessed: 19 March 2026).

Slotta, D. (2025) *Topic: Social media, Statista*. Available at: <https://www.statista.com/topics/1164/social-networks/> (Accessed: 22 March 2026).

Lazer, D. *et al.* (2014) 'The Parable of Google Flu: Traps in Big Data Analysis', *Science*, 343(6176), pp. 1203–1205. Available at: <https://doi.org/10.1126/science.1248506> .

Omoyiola, B.O. (2022) 'The social implications, risks, challenges and opportunities of big data', *Emerald Open Research*, 1(4). Available at: <https://doi.org/10.1108/EOR-04-2023-0014> .

What is Big Data? | IBM (2024). Available at: <https://www.ibm.com/think/topics/big-data> (Accessed: 19 March 2026).

TASK B:

Table design

For the data warehouse schema design, 3 tables were created which follow a very simple star-schema design as shown below:

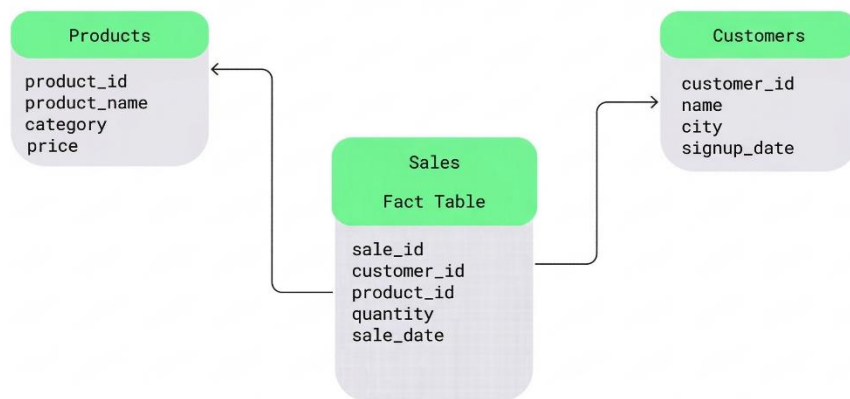


Figure 1: Data Warehous Schema Design

Star schema is a data modelling technique is adopted as in this context the data is organized into a central fact table (i.e., sales containing transactions) connected to multiple denormalized dimension tables (i.e., customers and products containing descriptive attributes like name, price, etc.)

Table Descriptions:

Table 1: Customers table

It is a dimension table that stores details about each customer. The customer_id is the primary key for this table. This column corresponds as the foreign key in the sales table through which different sales information about each customer can be joined and queried.

The data types are chosen in accordance to the fields they best represent. As customer_id for each customer is unique and represented by an integer number it has to be INT type. Similarly, name and city describe the name of the customer and their home

location(city) they belong to so, STRING type is the correct choice. Finally, a DATE type column is also present which effectively represents the date when the customer signed up or joined.

Table 1: Customer Table description

Columns	Datatype	Description
customer_id	INT	Unique identifier for each customer
name	STRING	Name of the customer
city	STRING	Location (City) of customer
signup_date	DATE	Date when the customer joined

Table 2: Products table

It is also a dimension table which stores product catalog information. The product_id is its unique product identifier which is its primary key, which later also corresponds as the foreign key in the sales table. This field allows sales table to link with it and find sales information for different products.

The product_id is again a unique integer number (INT) used for representing each product hence, no same products share the same ids. Name of the product is given by the product_name and also its category meaning what type of product it is, requires STRING data type. However, price of product can be in whole number but in decimals as well so to account for that DOUBLE data type is used.

Table 2: Products Table description

Columns	Data type	Description
product_id	INT	Unique identifier for each product
product_name	STRING	Name of the product
category	STRING	Type of product (e.g., electronics, furniture, etc.)
price	DOUBLE	Price of the product

Table 3: Sales table

It is the most important table in the data warehouse as it is the fact table which stores transaction data for different customers, the products they bought, the transaction id, how many units they bought and finally when they bought the product. The sale_id is the primary key of this table but it has customer_id and product_id as the foreign keys which provides a link or refer to the customers table as well as the products table. This allows

to interlink all the tables and make analysis on sales for each customer regarding what kind of products they purchased, when they purchased and how many.

The `sale_id` field represents a unique transaction so it is represented as integer. `customer_id` and `product_id` both are INT type from their original table. The quantity column expresses a certain quantity that cannot be in floating point so it should be an integer (INT). Finally, the `sale_date` is DATE type as it represents the date when the transaction was done.

Table 3: Sales Table description

Columns	Data type	Description
<code>sale_id</code>	INT	Unique identifier for each sale record
<code>customer_id</code>	INT	Reference to customers table
<code>product_id</code>	INT	Reference to products table
<code>quantity</code>	INT	Number of units purchased
<code>sale_date</code>	DATE	Transaction date

Building and Populating tables

First the Hive CLI is started and in the hive command prompt, the three tables are created using HiveQL statements.

Building tables:

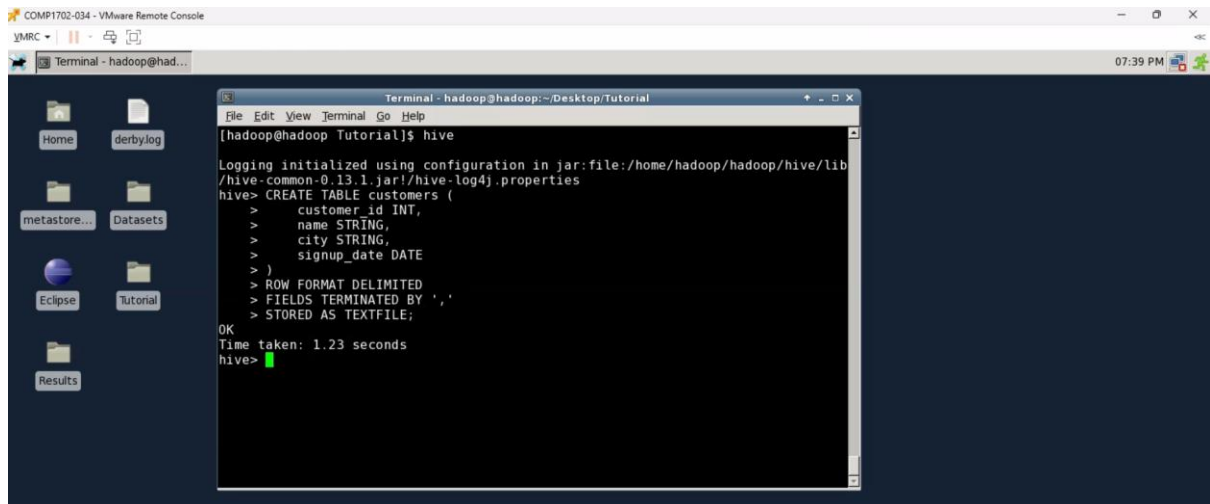
CREATE TABLE statement creates a table with:

- Table name (e.g. customers, products, sales),
- Column names and their types (e.g. id as INT, name as STRING),
- Input data format (e.g. separated by comma ‘,’)
- Input file structure(e.g. each record on a new line)

Query to create customers table:

```
CREATE TABLE customers (  
  customer_id INT,  
  name STRING,
```

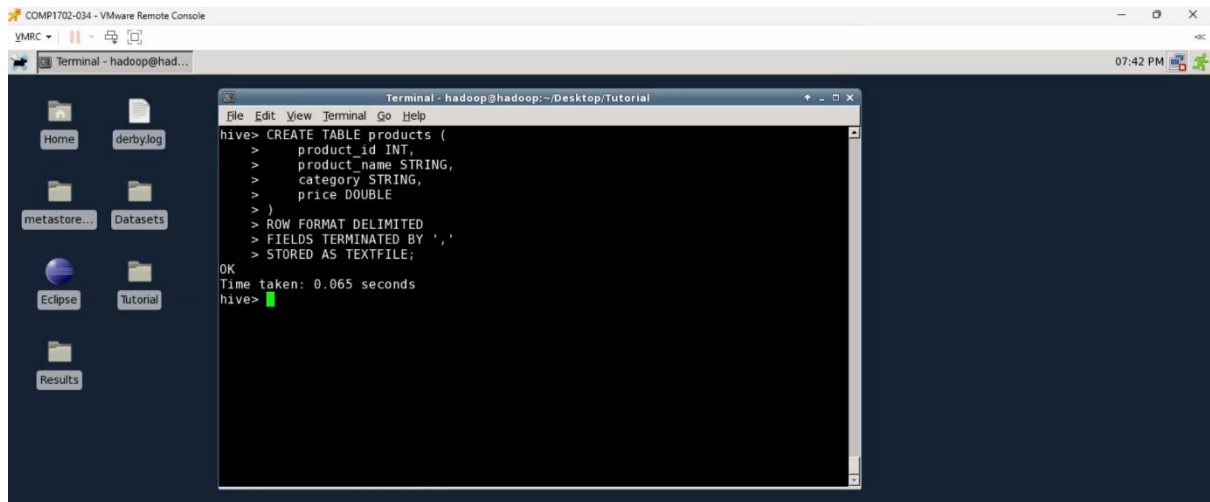
```
city STRING,  
signup_date DATE  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
STORED AS TEXTFILE;
```



A table named customers is created having columns customer_id, name, city and signup_date with data types as described earlier. ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' means that the data will be read from a comma-separated (CSV) file. And, STORED AS TEXTFILE instructs Hive to create table in text file format.

Query to create products table:

```
CREATE TABLE products (  
product_id INT,  
product_name STRING,  
category STRING,  
price DOUBLE  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
STORED AS TEXTFILE;
```



A table named products is created having columns product_id, product_name, category and price with data types as described earlier.

Query to create sales table:

```
CREATE TABLE sales (
```

```
    sale_id INT,
```

```
    customer_id INT,
```

```
    product_id INT,
```

```
    quantity INT,
```

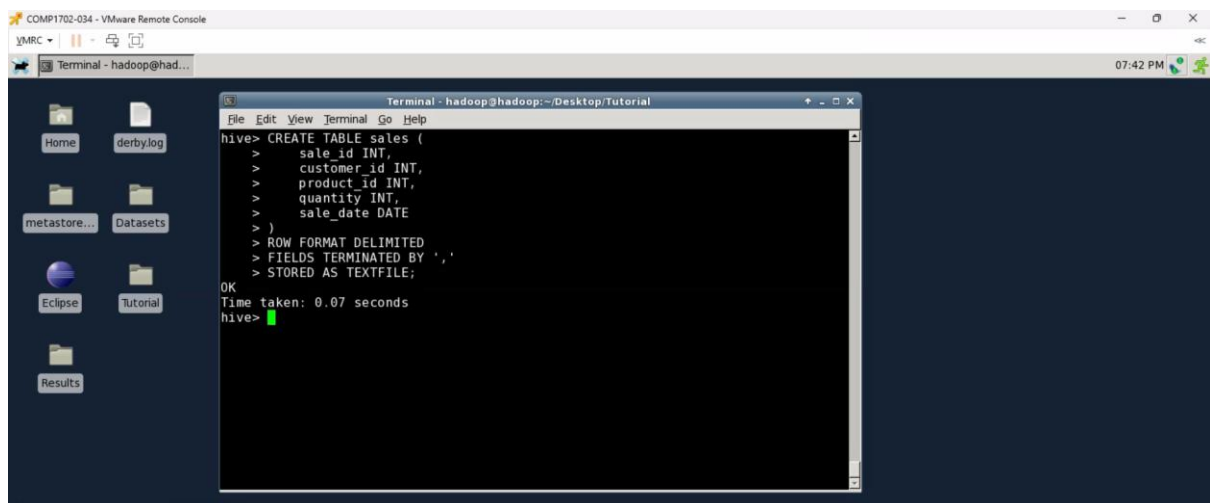
```
    sale_date DATE
```

```
)
```

```
ROW FORMAT DELIMITED
```

```
FIELDS TERMINATED BY ','
```

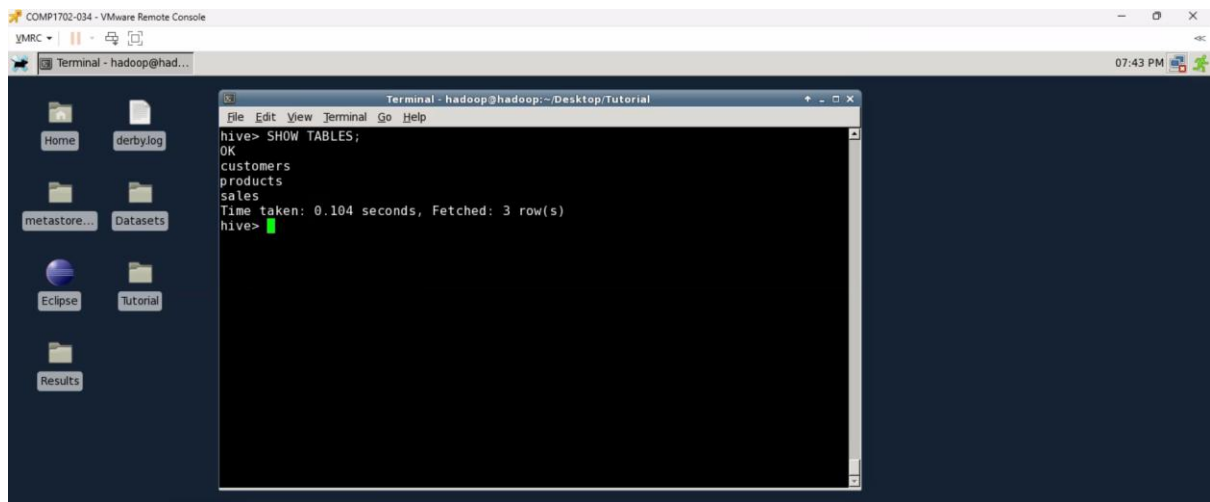
```
STORED AS TEXTFILE;
```



A table named sales is created having columns sale_id, customer_id, product_id, quantity and sale_date with data types as described earlier.

A quick sanity check to see if the tables are created can be done by:

SHOW TABLES;

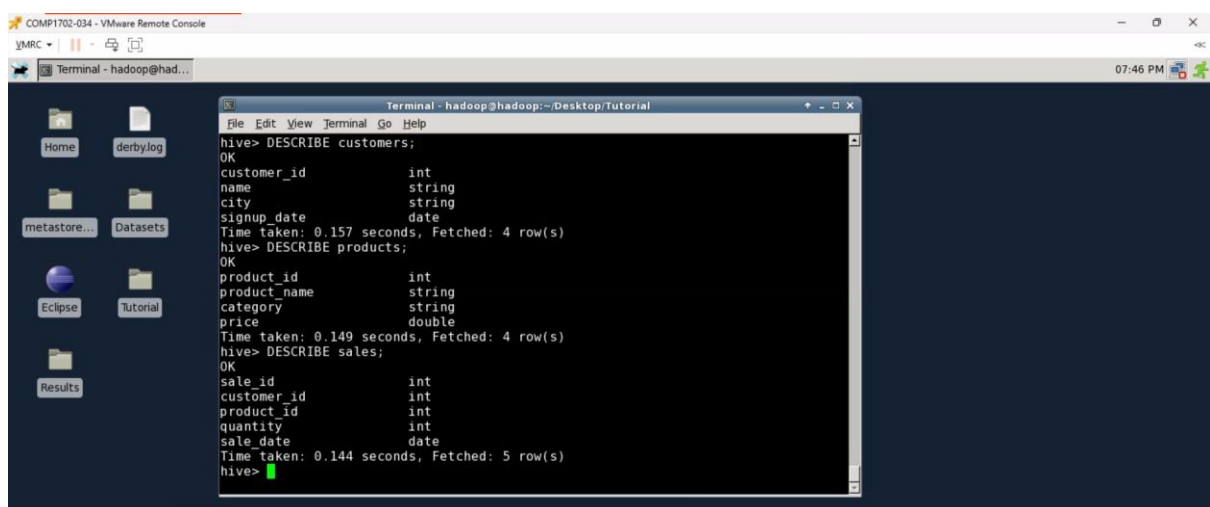


This confirms that the tables were indeed created as desired. But it can be taken a step further to make sure that the columns for each table are created with the correct data type, this can also be checked using: DESCRIBE table_name; (replacing table_name with the name of the tables created).

DESCRIBE customers;

DESCRIBE products;

DESCRIBE sales;

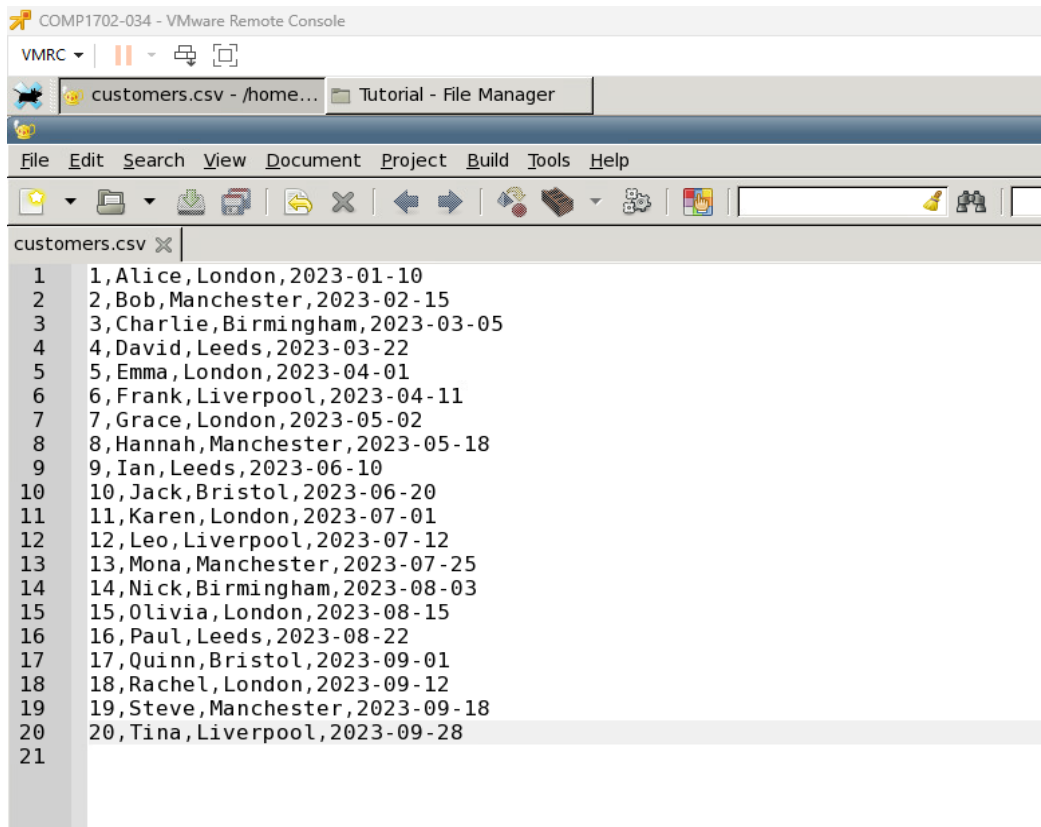


The DESCRIBE command in Hive is used to get information about a database, table, view, or columns. It is a quick way to see the structure of a table (columns, data types). It can be confirmed that all the table also have correct datatypes.

Populating tables

To hold the coursework datasets, file named Tutorial was created. They are named as customers.csv, products.csv and sales.csv.

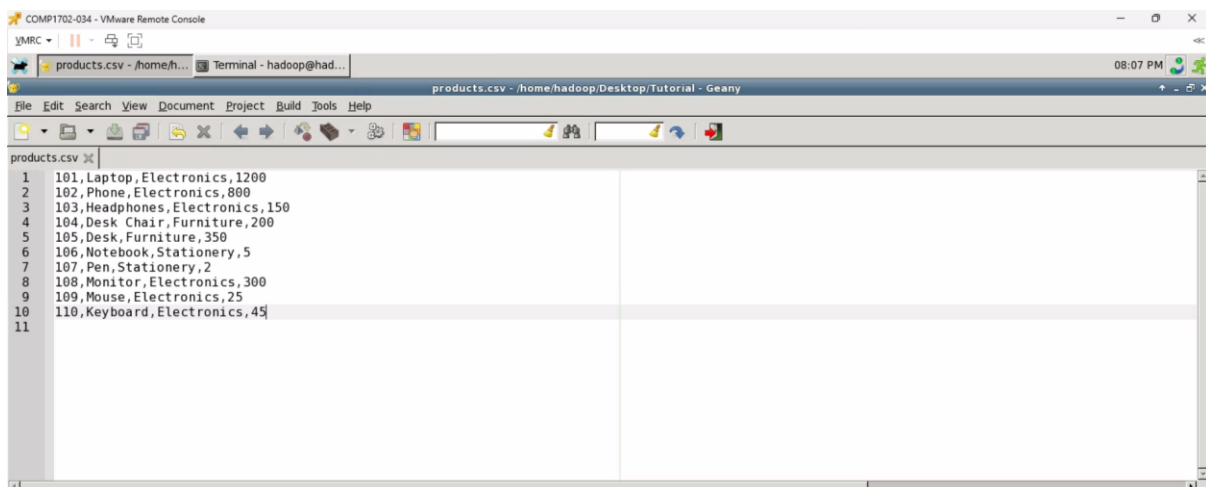
customers.csv



```
customers.csv
1 1,Alice,London,2023-01-10
2 2,Bob,Manchester,2023-02-15
3 3,Charlie,Birmingham,2023-03-05
4 4,David,Leeds,2023-03-22
5 5,Emma,London,2023-04-01
6 6,Frank,Liverpool,2023-04-11
7 7,Grace,London,2023-05-02
8 8,Hannah,Manchester,2023-05-18
9 9,Ian,Leeds,2023-06-10
10 10,Jack,Bristol,2023-06-20
11 11,Karen,London,2023-07-01
12 12,Leo,Liverpool,2023-07-12
13 13,Mona,Manchester,2023-07-25
14 14,Nick,Birmingham,2023-08-03
15 15,Olivia,London,2023-08-15
16 16,Paul,Leeds,2023-08-22
17 17,Quinn,Bristol,2023-09-01
18 18,Rachel,London,2023-09-12
19 19,Steve,Manchester,2023-09-18
20 20,Tina,Liverpool,2023-09-28
21
```

The customers.csv file holds the customer data which includes 20 records.

products.csv



```
products.csv
1 101,Laptop,Electronics,1200
2 102,Phone,Electronics,800
3 103,Headphones,Electronics,150
4 104,Desk Chair,Furniture,200
5 105,Desk,Furniture,350
6 106,Notebook,Stationery,5
7 107,Pen,Stationery,2
8 108,Monitor,Electronics,300
9 109,Mouse,Electronics,25
10 110,Keyboard,Electronics,45
11
```

The products.csv file holds the product information which includes 10 records.

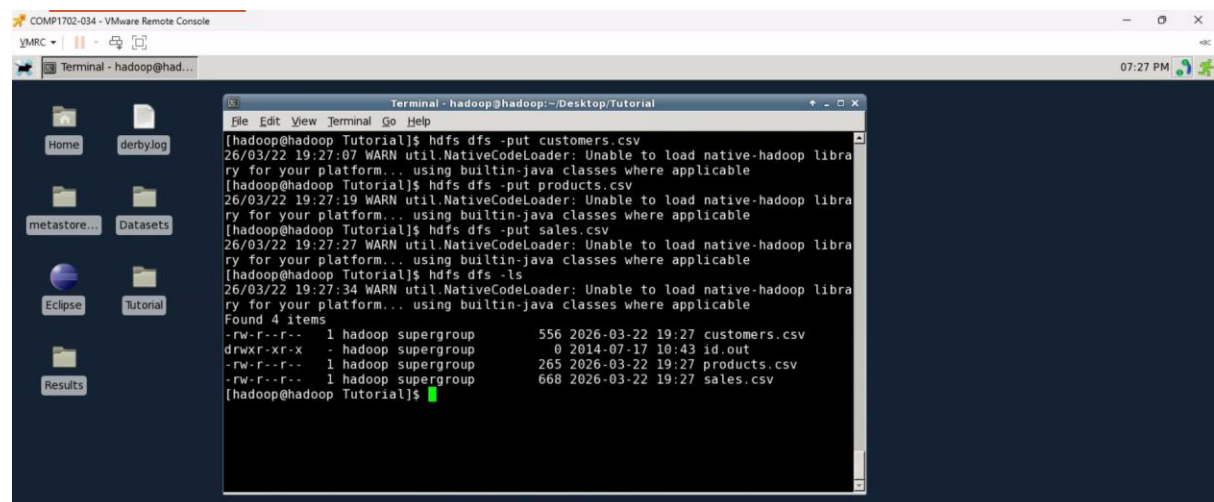
sales.csv

The image shows a VMware Remote Console window. At the top, the title bar reads "COMPT1702-034 - VMware Remote Console". Below the title bar is a toolbar with icons for VMRC, a power button, a refresh icon, and a window icon. The main window has a tab labeled "sales.csv - /home/hado..." and another tab labeled "Tutorial - File Manager". The "sales.csv" tab is active, showing a file editor with a menu bar (File, Edit, Search, View, Document, Project, Build, Tools, Help) and a toolbar with various icons. The file content is a CSV with 30 rows of data, each starting with a line number from 1 to 30. The data consists of five columns: two integers, a string, another integer, and a date in YYYY-MM-DD format. The data is as follows:

1	1	1	101	1	2024-01-01
2	2	2	102	2	2024-01-02
3	3	3	103	3	2024-01-02
4	4	4	104	1	2024-01-03
5	5	5	105	1	2024-01-04
6	6	6	106	10	2024-01-05
7	7	7	107	20	2024-01-06
8	8	8	108	1	2024-01-07
9	9	9	109	2	2024-01-07
10	10	10	110	1	2024-01-08
11	11	1	102	1	2024-01-09
12	12	2	103	2	2024-01-10
13	13	3	104	1	2024-01-11
14	14	4	105	1	2024-01-12
15	15	5	106	5	2024-01-13
16	16	6	107	10	2024-01-14
17	17	7	108	1	2024-01-15
18	18	8	109	1	2024-01-16
19	19	9	110	1	2024-01-17
20	20	10	101	1	2024-01-18
21	21	11	102	1	2024-01-19
22	22	12	103	2	2024-01-20
23	23	13	104	1	2024-01-21
24	24	14	105	1	2024-01-22
25	25	15	106	10	2024-01-23
26	26	16	107	15	2024-01-24
27	27	17	108	1	2024-01-25
28	28	18	109	2	2024-01-26
29	29	19	110	1	2024-01-27
30	30	20	101	1	2024-01-28

The sales.csv file holds the transaction information for every customer and their purchased products which includes a total of 30 records.

Before populating the tables with data, the data files need to be uploaded in HDFS. This can be done by:



```
COMP1702-034 - VMware Remote Console
VMRC | | | 
sales.csv - /home/hado... Tutorial - File Manager
File Edit Search View Document Project Build Tools Help
sales.csv x
1 1,1,101,1,2024-01-01
2 2,2,102,2,2024-01-02
3 3,3,103,3,2024-01-02
4 4,4,104,1,2024-01-03
5 5,5,105,1,2024-01-04
6 6,6,106,10,2024-01-05
7 7,7,107,20,2024-01-06
8 8,8,108,1,2024-01-07
9 9,9,109,2,2024-01-07
10 10,10,110,1,2024-01-08
11 11,1,102,1,2024-01-09
12 12,2,103,2,2024-01-10
13 13,3,104,1,2024-01-11
14 14,4,105,1,2024-01-12
15 15,5,106,5,2024-01-13
16 16,6,107,10,2024-01-14
17 17,7,108,1,2024-01-15
18 18,8,109,1,2024-01-16
19 19,9,110,1,2024-01-17
20 20,10,101,1,2024-01-18
21 21,11,102,1,2024-01-19
22 22,12,103,2,2024-01-20
23 23,13,104,1,2024-01-21
24 24,14,105,1,2024-01-22
25 25,15,106,10,2024-01-23
26 26,16,107,15,2024-01-24
27 27,17,108,1,2024-01-25
28 28,18,109,2,2024-01-26
29 29,19,110,1,2024-01-27
30 30,20,101,1,2024-01-28
31

The sales.csv file holds the transaction information for every customer and their purchased products which includes a total of 30 records.

Before populating the tables with data, the data files need to be uploaded in HDFS. This can be done by:

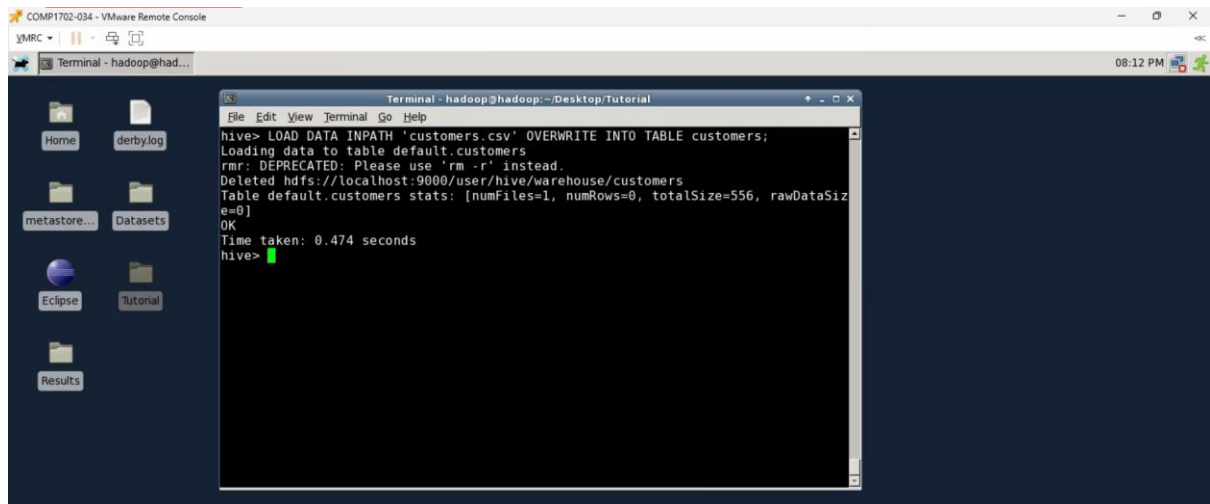
COMP1702-034 - VMware Remote Console
VMRC | | | 
Terminal - hadoop@hadoop... 07:27 PM
Home derbylog
metastore... Datasets
Eclipse Tutorial
Results
Terminal - hadoop@hadoop:~/Desktop/Tutorial
File Edit View Terminal Go Help
[hadoop@hadoop Tutorial]$ hdfs dfs -put customers.csv
26/03/22 19:27:07 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
[hadoop@hadoop Tutorial]$ hdfs dfs -put products.csv
26/03/22 19:27:19 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
[hadoop@hadoop Tutorial]$ hdfs dfs -put sales.csv
26/03/22 19:27:27 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
[hadoop@hadoop Tutorial]$ hdfs dfs -ls
26/03/22 19:27:34 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Found 4 items
-rw-r--r-- 1 hadoop supergroup 556 2026-03-22 19:27 customers.csv
drwxr-xr-x 1 hadoop supergroup 0 2014-07-17 10:43 id.out
-rw-r--r-- 1 hadoop supergroup 265 2026-03-22 19:27 products.csv
-rw-r--r-- 1 hadoop supergroup 668 2026-03-22 19:27 sales.csv
[hadoop@hadoop Tutorial]$
```

Once the file gets uploaded in the HDFS, the Load statement can be used to load data (stored on HDFS) into the hive table using:

LOAD DATA INPATH `hdfs_file` OVERWRITE INTO TABLE `table_name`;

Loading data to the customers table:

LOAD DATA INPATH 'customers.csv' OVERWRITE INTO TABLE customers;



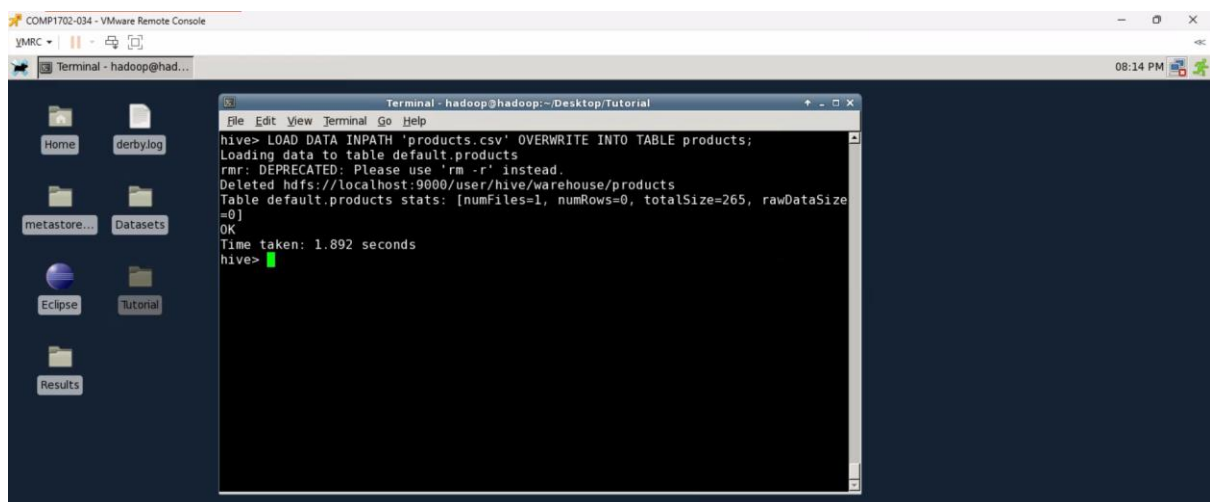
The screenshot shows a terminal window titled 'Terminal - hadoop@hadoop:~/Desktop/Tutorial'. The terminal displays the following output for the command `hive> LOAD DATA INPATH 'customers.csv' OVERWRITE INTO TABLE customers;`:

```
hive> LOAD DATA INPATH 'customers.csv' OVERWRITE INTO TABLE customers;
Loading data to table default.customers
rmr: DEPRECATED: Please use 'rm -r' instead.
Deleted hdfs://localhost:9000/user/hive/warehouse/customers
Table default.customers stats: [numFiles=1, numRows=0, totalSize=556, rawDataSize=0]
OK
Time taken: 0.474 seconds
hive>
```

This loads the data from 'customers.csv' file which was uploaded in HDFS directory into the hive table named customers.

Loading data to the products table:

LOAD DATA INPATH 'products.csv' OVERWRITE INTO TABLE products;



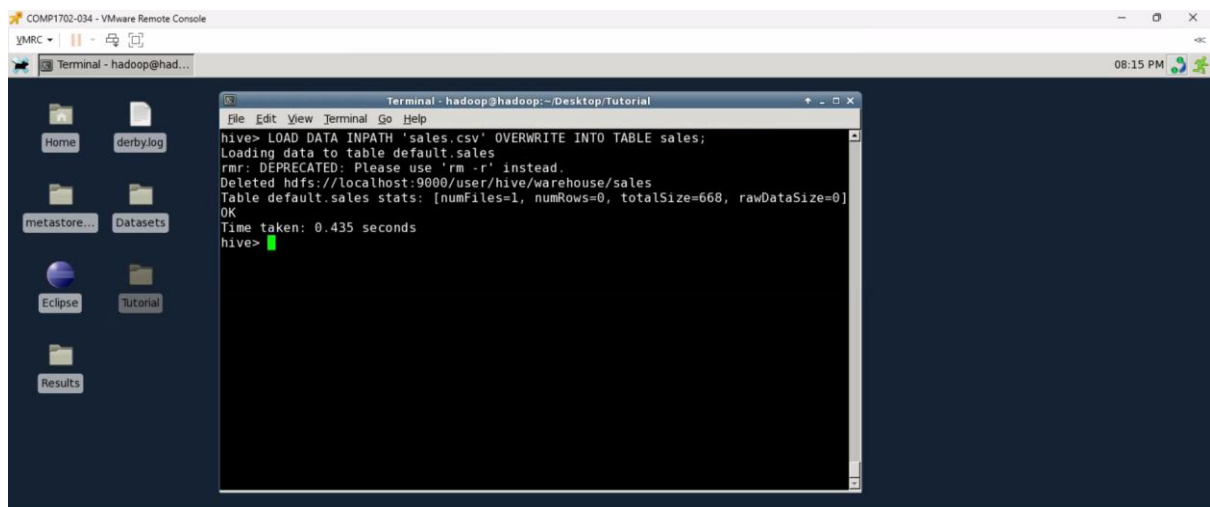
The screenshot shows a terminal window titled 'Terminal - hadoop@hadoop:~/Desktop/Tutorial'. The terminal displays the following output for the command `hive> LOAD DATA INPATH 'products.csv' OVERWRITE INTO TABLE products;`:

```
hive> LOAD DATA INPATH 'products.csv' OVERWRITE INTO TABLE products;
Loading data to table default.products
rmr: DEPRECATED: Please use 'rm -r' instead.
Deleted hdfs://localhost:9000/user/hive/warehouse/products
Table default.products stats: [numFiles=1, numRows=0, totalSize=265, rawDataSize=0]
OK
Time taken: 1.892 seconds
hive>
```

This loads the data from 'products.csv' file which was uploaded in HDFS directory into the hive table named products.

Loading data to the sales table:

LOAD DATA INPATH 'sales.csv' OVERWRITE INTO TABLE sales;



This loads the data from 'sales.csv' file which was uploaded in HDFS directory into the hive table named sales.

After all the necessary tables are built and populated with their respective records, HiveQL can be used to query the tables.

Query Design

Query-1: Retrieve detailed sales transactions

Query:

```
SELECT s.sale_id, c.name AS customer_name, p.product_name, s.quantity, s.sale_date
FROM sales s
```

```
JOIN customers c
```

```
ON s.customer_id = c.customer_id
```

```
JOIN products p
```

```
ON s.product_id = p.product_id;
```

This is a data retrieval query which retrieves a list of sales transactions along with associated customer and product details. It joins the sales, customers, and products tables using their respective IDs to produce a combined view that includes the sale ID, customer name, product name, quantity sold, and sale date. By default, it is an INNER JOIN so it outputs only records with matching entries across all three tables are included in the result.

Execution:

```
COMP1702-034 - VMware Remote Console
VMRC
File Edit View Terminal Go Help
hive> SELECT
> s.sale_id,
> c.name AS customer_name,
> p.product_name,
> s.quantity,
> s.sale_date
> FROM sales s
> JOIN customers c
> ON s.customer_id = c.customer_id
> JOIN products p
> ON s.product_id = p.product_id;
Total jobs = 1
26/03/22 20:25:48 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
26/03/22 20:25:48 WARN conf.Configuration: file:/tmp/hadoop/hive_2026-03-22_20-25-44_002_5263635051167183497-1/-local-10007/jobconf.xml:an attempt to o
verride final parameter: mapreduce.job.end-notification.max.retry.interval; Ignoring.
26/03/22 20:25:48 WARN conf.Configuration: file:/tmp/hadoop/hive_2026-03-22_20-25-44_002_5263635051167183497-1/-local-10007/jobconf.xml:an attempt to o
verride final parameter: mapreduce.job.end-notification.max.attempts; Ignoring.
Execution log at: /tmp/hadoop/hadoop_20260322202525_851b08e2-b9b1-4f97-822d-58c6f0342746.log
2026-03-22 08:25:48 Starting to launch local task to process map join; maximum memory = 518979584
2026-03-22 08:25:49 Dump the side-table into file: file:/tmp/hadoop/hive_2026-03-22_20-25-44_002_5263635051167183497-1/-local-10004/HashTable-Stage
-4/MapJoin-mapfile01--.hashtable
2026-03-22 08:25:49 Uploaded 1 File to: file:/tmp/hadoop/hive_2026-03-22_20-25-44_002_5263635051167183497-1/-local-10004/HashTable-Stage-4/MapJoin-
mapfile01--.hashtable (529 bytes)
2026-03-22 08:25:49 Dump the side-table into file: file:/tmp/hadoop/hive_2026-03-22_20-25-44_002_5263635051167183497-1/-local-10004/HashTable-Stage
-4/MapJoin-mapfile11--.hashtable
2026-03-22 08:25:49 Uploaded 1 File to: file:/tmp/hadoop/hive_2026-03-22_20-25-44_002_5263635051167183497-1/-local-10004/HashTable-Stage-4/MapJoin-
mapfile11--.hashtable (756 bytes)
2026-03-22 08:25:49 End of local task; Time Taken: 0.651 sec.
Execution completed successfully
```

Output:

```
COMP1702-034 - VMware Remote Console
VMRC
File Edit View Terminal Go Help
Job 08 Maps: 1 Cumulative CPU: 0.85 sec HDFS Read: 879 HDFS Write: 874 SUCCESS
Total MapReduce CPU Time Spent: 850 msec
OK
1 Alice Laptop 1 2024-01-01
2 Bob Phone 2 2024-01-02
3 Charlie Headphones 3 2024-01-02
4 David Desk Chair 1 2024-01-03
5 Emma Desk 1 2024-01-04
6 Frank Notebook 10 2024-01-05
7 Grace Pen 20 2024-01-06
8 Hannah Monitor 1 2024-01-07
9 Ian Mouse 2 2024-01-07
10 Jack Keyboard 1 2024-01-08
11 Alice Phone 1 2024-01-09
12 Bob Headphones 2 2024-01-10
13 Charlie Desk Chair 1 2024-01-11
14 David Desk 1 2024-01-12
15 Emma Notebook 5 2024-01-13
16 Frank Pen 10 2024-01-14
17 Grace Monitor 1 2024-01-15
18 Hannah Mouse 1 2024-01-16
19 Ian Keyboard 1 2024-01-17
20 Jack Laptop 1 2024-01-18
21 Karen Phone 1 2024-01-19
22 Leo Headphones 2 2024-01-20
23 Mona Desk Chair 1 2024-01-21
24 Nick Desk 1 2024-01-22
25 Olivia Notebook 10 2024-01-23
26 Paul Pen 15 2024-01-24
27 Quinn Monitor 1 2024-01-25
28 Rachel Mouse 2 2024-01-26
29 Steve Keyboard 1 2024-01-27
30 Tina Laptop 1 2024-01-28
Time taken: 19.879 seconds, Fetched: 30 row(s)
hive>
```

The output shows a single view made by joining all three tables with records showing sales id for the transaction they performed, the customer's name who bought the product, product name as in which product they bought, quantity for how many they bought and finally signup date as in when they bought.

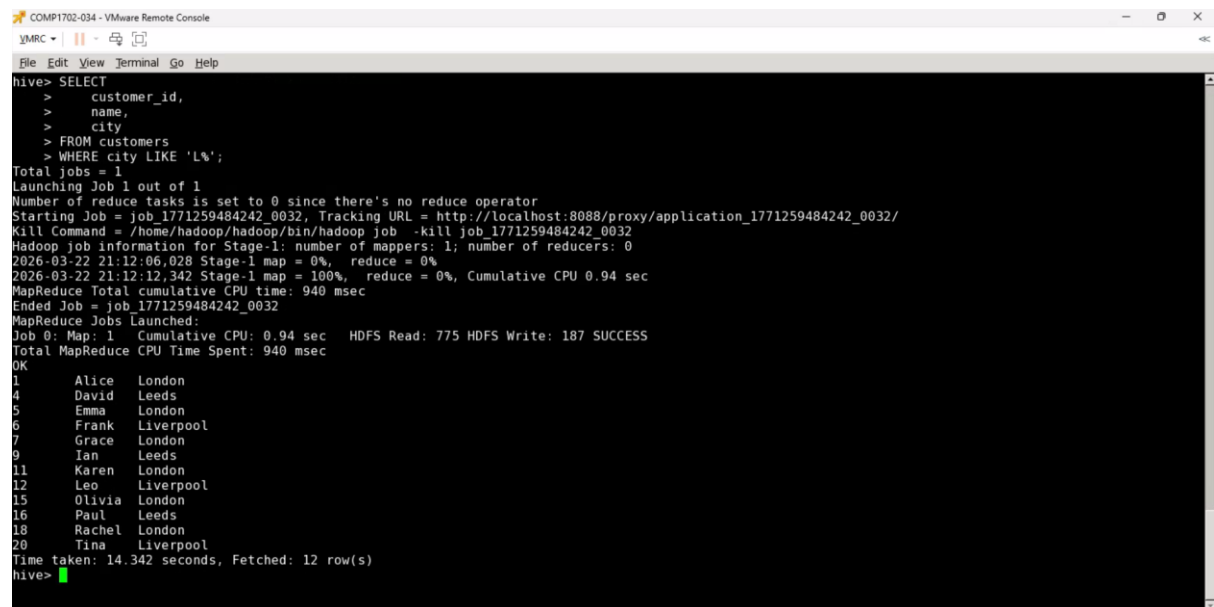
Query-2: Retrieve customers from city starting from 'L'

Query:

```
SELECT customer_id, name, city
FROM customers
WHERE city LIKE 'L%';
```

This is a data retrieval query which selects customer records from the customers table where the city name begins with the letter 'L'. It uses the LIKE operator with a wildcard (%) to filter cities that start with 'L', returning the customer ID, name, and city.

Execution and Output:



```
hive> SELECT
>   customer_id,
>   name,
>   city
> FROM customers
> WHERE city LIKE 'L%';
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1771259484242_0032, Tracking URL = http://localhost:8088/proxy/application_1771259484242_0032/
Kill Command = /home/hadoop/hadoop/bin/hadoop job -kill job_1771259484242_0032
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 0
2026-03-22 21:12:06,028 Stage-1 map = 0%, reduce = 0%
2026-03-22 21:12:12,342 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 0.94 sec
MapReduce Total cumulative CPU time: 940 msec
Ended Job = job_1771259484242_0032
MapReduce Jobs Launched:
Job 0: Map: 1 Cumulative CPU: 0.94 sec HDFS Read: 775 HDFS Write: 187 SUCCESS
Total MapReduce CPU Time Spent: 940 msec
OK
1      Alice      London
4      David      Leeds
5      Emma       London
6      Frank      Liverpool
7      Grace      London
9      Ian        Leeds
11     Karen      London
12     Leo        Liverpool
15     Olivia     London
16     Paul       Leeds
18     Rachel     London
20     Tina       Liverpool
Time taken: 14.342 seconds, Fetched: 12 row(s)
hive>
```

The output returns a series of rows including customer ID, customer name and their cities which can be seen all start from 'L' satisfying the needed condition.

Query-3: Retrieve customers who live in London and bought a Laptop before 2024-01-10

Query:

```
SELECT
    c.customer_id AS customer_id,
    c.name AS customer_name,
    c.city AS city,
    p.product_name AS product_name,
    s.sale_date AS purchase_date
FROM customers c
JOIN sales s
ON c.customer_id = s.customer_id
```


JOIN products p

ON s.product_id = p.product_id

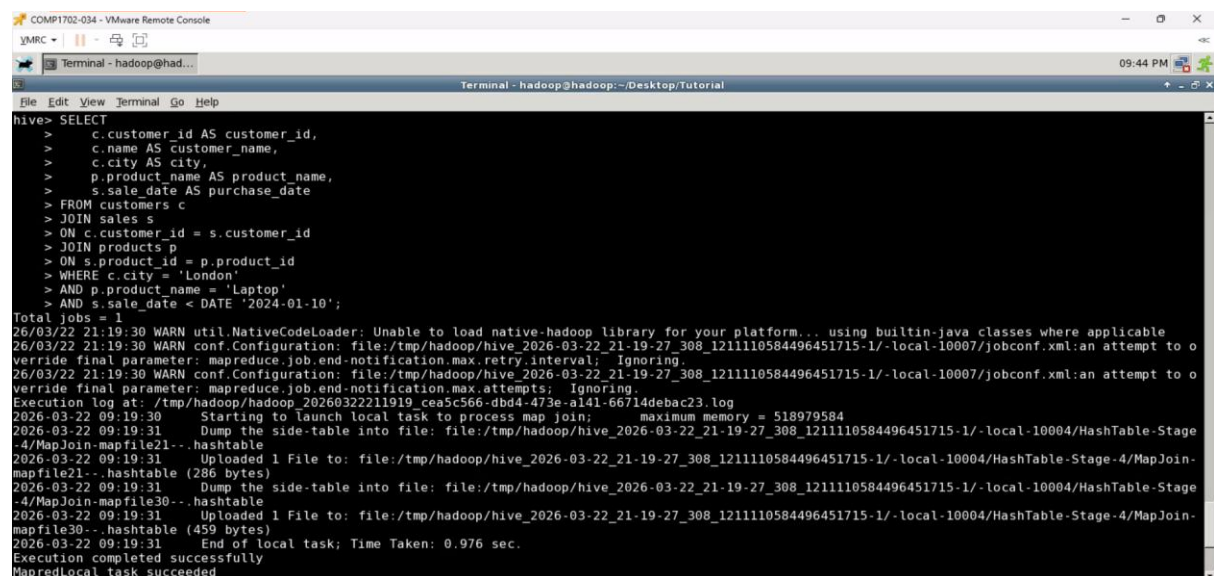
WHERE c.city = 'London'

AND p.product_name = 'Laptop'

AND s.sale_date < DATE '2024-01-10';

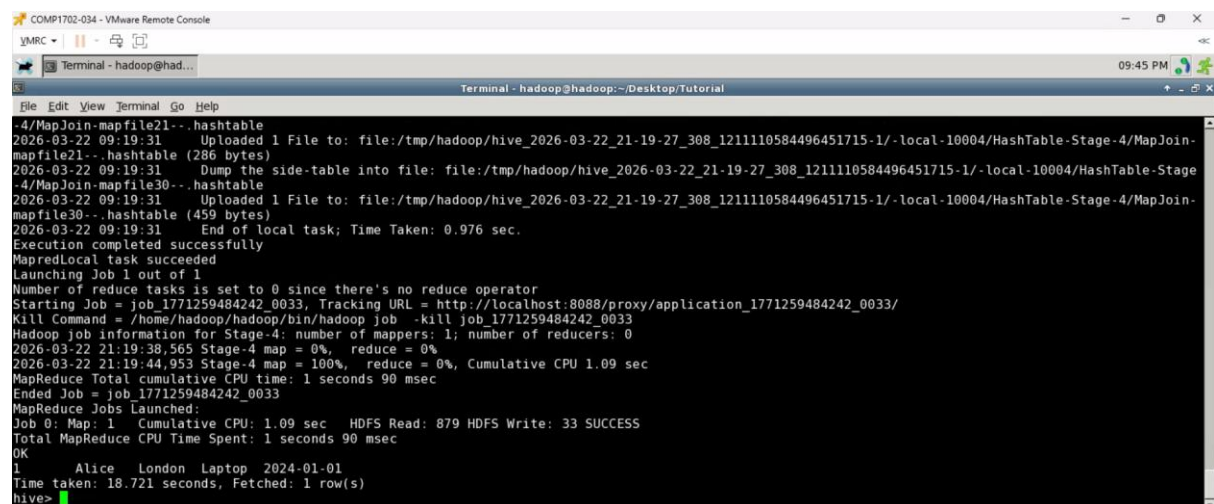
This is data retrieval query which retrieves customer purchase records by joining the customers, sales, and products tables. It then filters the results to include only customers located in London who purchased a product named 'Laptop' before 10th January 2024.

Execution:



```
hive> SELECT
>   c.customer_id AS customer_id,
>   c.name AS customer_name,
>   c.city AS city,
>   p.product_name AS product_name,
>   s.sale_date AS purchase_date
> FROM customers c
> JOIN sales s
> ON c.customer_id = s.customer_id
> JOIN products p
> ON s.product_id = p.product_id
> WHERE c.city = 'London'
> AND p.product_name = 'Laptop'
> AND s.sale_date < DATE '2024-01-10';
Total jobs = 1
26/03/22 21:19:30 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
26/03/22 21:19:30 WARN conf.Configuration: file:/tmp/hadoop/hive_2026-03-22_21-19-27_308_121110584496451715-1/-local-10007/jobconf.xml:an attempt to o
verride final parameter: mapreduce.job.end-notification.max.retry.interval; Ignoring.
26/03/22 21:19:30 WARN conf.Configuration: file:/tmp/hadoop/hive_2026-03-22_21-19-27_308_121110584496451715-1/-local-10007/jobconf.xml:an attempt to o
verride final parameter: mapreduce.job.end-notification.max.attempts; Ignoring.
Execution log at: /tmp/hadoop/hadoop_20260322211919_ces5c566-dbd4-473e-a141-66714deba23.log
2026-03-22 09:19:30 Starting to launch local task to process map join; maximum memory = 518979584
2026-03-22 09:19:31 Dump the side-table into file: file:/tmp/hadoop/hive_2026-03-22_21-19-27_308_121110584496451715-1/-local-10004/HashTable-Stage
-4/MapJoin-mapfile21--.hashtable
2026-03-22 09:19:31 Uploaded 1 File to: file:/tmp/hadoop/hive_2026-03-22_21-19-27_308_121110584496451715-1/-local-10004/HashTable-Stage-4/MapJoin-
mapfile21--.hashtable (286 bytes)
2026-03-22 09:19:31 Dump the side-table into file: file:/tmp/hadoop/hive_2026-03-22_21-19-27_308_121110584496451715-1/-local-10004/HashTable-Stage
-4/MapJoin-mapfile30--.hashtable
2026-03-22 09:19:31 Uploaded 1 File to: file:/tmp/hadoop/hive_2026-03-22_21-19-27_308_121110584496451715-1/-local-10004/HashTable-Stage-4/MapJoin-
mapfile30--.hashtable (459 bytes)
2026-03-22 09:19:31 End of local task; Time Taken: 0.976 sec.
Execution completed successfully
MapredLocal task succeeded
```

Output:



```
2026-03-22 09:19:31 Uploaded 1 File to: file:/tmp/hadoop/hive_2026-03-22_21-19-27_308_121110584496451715-1/-local-10004/HashTable-Stage-4/MapJoin-
mapfile21--.hashtable (286 bytes)
2026-03-22 09:19:31 Dump the side-table into file: file:/tmp/hadoop/hive_2026-03-22_21-19-27_308_121110584496451715-1/-local-10004/HashTable-Stage
-4/MapJoin-mapfile30--.hashtable
2026-03-22 09:19:31 Uploaded 1 File to: file:/tmp/hadoop/hive_2026-03-22_21-19-27_308_121110584496451715-1/-local-10004/HashTable-Stage-4/MapJoin-
mapfile30--.hashtable (459 bytes)
2026-03-22 09:19:31 End of local task; Time Taken: 0.976 sec.
Execution completed successfully
MapredLocal task succeeded
Launching Job 1 out of 1
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1771259484242_0033, Tracking URL = http://localhost:8088/proxy/application_1771259484242_0033/
Kill Command = /home/hadoop/hadoop/bin/hadoop job -kill job_1771259484242_0033
Hadoop job information for Stage-4: number of mappers: 1; number of reducers: 0
2026-03-22 21:19:38,565 Stage-4 map = 0%, reduce = 0%
2026-03-22 21:19:44,953 Stage-4 map = 100%, reduce = 0%, Cumulative CPU 1.09 sec
MapReduce Total cumulative CPU time: 1 seconds 90 msec
Ended Job = job_1771259484242_0033
MapReduce Jobs Launched:
Job 0: Map: 1 Cumulative CPU: 1.09 sec HDFS Read: 879 HDFS Write: 33 SUCCESS
Total MapReduce CPU Time Spent: 1 seconds 90 msec
OK
1 Alice London Laptop 2024-01-01
Time taken: 18.721 seconds, Fetched: 1 row(s)
hive>
```


The output shows customer ID, customer name, city, product name, and purchase date for people who match all of the conditions, for this case being only one customer named 'Alice'.

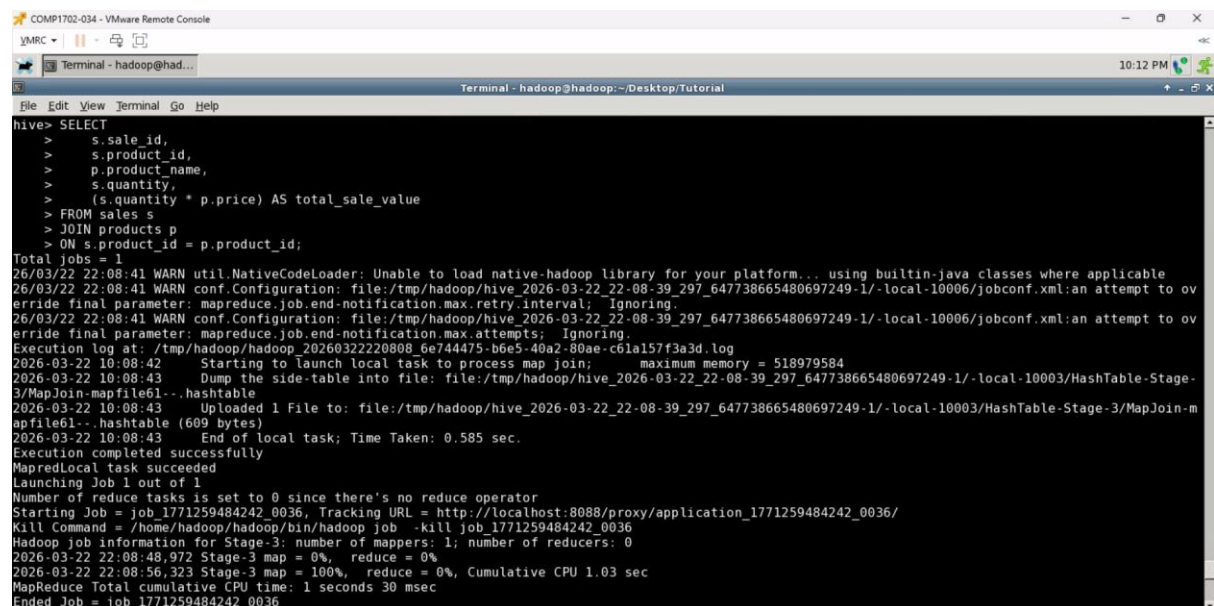
Query-4: Calculates total revenue generated from each sale

Query:

```
SELECT
    s.sale_id,
    s.product_id,
    p.product_name,
    s.quantity,
    (s.quantity * p.price) AS total_sale_value
FROM sales s
JOIN products p
ON s.product_id = p.product_id;
```

This is a data manipulation query which retrieves sales data by joining the sales and products tables using the product ID. It calculates the total revenue generated by each sale by multiplying the quantity sold by the product price.

Execution:



```
COMP1702-034 - VMware Remote Console
VMRC
Terminal - hadoop@hadoop:~/Desktop/Tutorial
10:12 PM
File Edit View Terminal Go Help
hive> SELECT
>   s.sale_id,
>   s.product_id,
>   p.product_name,
>   s.quantity,
>   (s.quantity * p.price) AS total_sale_value
> FROM sales s
> JOIN products p
> ON s.product_id = p.product_id;
Total jobs = 1
26/03/22 22:08:41 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
26/03/22 22:08:41 WARN conf.Configuration: file:/tmp/hadoop/hive_2026-03-22_22-08-39_297_647738665480697249-1/-local-10006/jobconf.xml:an attempt to ov
erride final parameter: mapreduce.job.end-notification.max.retry.interval; Ignoring.
26/03/22 22:08:41 WARN conf.Configuration: file:/tmp/hadoop/hive_2026-03-22_22-08-39_297_647738665480697249-1/-local-10006/jobconf.xml:an attempt to ov
erride final parameter: mapreduce.job.end-notification.max.attempts; Ignoring.
Execution log at: /tmp/hadoop/hadoop_20260322220808_6e744475-b6e5-40a2-80ae-c61a157f3a3d.log
2026-03-22 10:08:42 Starting to launch local task to process map join; maximum memory = 518979584
2026-03-22 10:08:43 Dump the side-table into file: file:/tmp/hadoop/hive_2026-03-22_22-08-39_297_647738665480697249-1/-local-10003/HashTable-Stage-
3/MapJoin-mapfile61--..hashtable
2026-03-22 10:08:43 Uploaded 1 File to: file:/tmp/hadoop/hive_2026-03-22_22-08-39_297_647738665480697249-1/-local-10003/HashTable-Stage-3/MapJoin-m
apfile61--..hashtable (609 bytes)
2026-03-22 10:08:43 End of local task; Time Taken: 0.585 sec.
Execution completed successfully
MapredLocal task succeeded
Launching Job 1 out of 1
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1771259484242_0036, Tracking URL = http://localhost:8088/proxy/application_1771259484242_0036/
Kill Command = /home/hadoop/hadoop/bin/hadoop job -kill job_1771259484242_0036
Hadoop job information for Stage-3: number of mappers: 1; number of reducers: 0
2026-03-22 22:08:48,972 Stage-3 map = 0%, reduce = 0%
2026-03-22 22:08:56,323 Stage-3 map = 100%, reduce = 0%, Cumulative CPU 1.03 sec
MapReduce Total cumulative CPU time: 1 seconds 30 msec
Ended Job = job_1771259484242_0036
```

Output:

```
OK
1      101      Laptop  1      1200.0
2      102      Phone   2      1600.0
3      103      Headphones 3      450.0
4      104      Desk Chair 1      200.0
5      105      Desk     1      350.0
6      106      Notebook  10     50.0
7      107      Pen       20     40.0
8      108      Monitor   1      300.0
9      109      Mouse     2      50.0
10     110      Keyboard  1      45.0
11     102      Phone   1      800.0
12     103      Headphones 2      300.0
13     104      Desk Chair 1      200.0
14     105      Desk     1      350.0
15     106      Notebook  5      25.0
16     107      Pen       10     20.0
17     108      Monitor   1      300.0
18     109      Mouse     1      25.0
19     110      Keyboard  1      45.0
20     101      Laptop  1      1200.0
21     102      Phone   1      800.0
22     103      Headphones 2      300.0
23     104      Desk Chair 1      200.0
24     105      Desk     1      350.0
25     106      Notebook  10     50.0
26     107      Pen       15     30.0
27     108      Monitor   1      300.0
28     109      Mouse     2      50.0
29     110      Keyboard  1      45.0
30     101      Laptop  1      1200.0
Time taken: 18.119 seconds, Fetched: 30 row(s)
hive>
```

The output shows which sales as per their IDs, what product was bought, quantity and the total sale value for that transaction.

Query-5: Show Discounted price of Products using CTE

Query:

WITH discounted_products AS (

SELECT product_id, product_name, category, price, price * 0.9 AS discounted_price

FROM products

)

SELECT product_id, product_name, category, price, discounted_price

FROM discounted_products;

This query is a data manipulation query which uses a Common Table Expression (CTE) to first create a temporary result set and then retrieve data from it, to calculate discounted product prices. The CTE generates a temporary result set containing product details along with a 10% discounted price. The main query then selects and displays all fields from this intermediate result, improving clarity and structure of the query.

Execution:

```
COMP1702-034 - VMware Remote Console
VMRC
Terminal - hadoop@had...
Terminal - hadoop@hadoop:~/Desktop/Tutorial

hive> WITH discounted_products AS (
>   SELECT
>     product_id,
>     product_name,
>     category,
>     price,
>     price * 0.9 AS discounted_price
>   FROM products
> )
> SELECT
>   product_id,
>   product_name,
>   category,
>   price,
>   discounted_price
> FROM discounted_products;
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1771259484242_0038, Tracking URL = http://localhost:8088/proxy/application_1771259484242_0038/
Kill Command = /home/hadoop/hadoop/bin/hadoop job -kill job_1771259484242_0038
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 0
2026-03-22 22:37:08,522 Stage-1 map = 0%, reduce = 0%
2026-03-22 22:37:14,768 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 0.91 sec
MapReduce Total cumulative CPU time: 910 msec
```

Output:

```
COMP1702-034 - VMware Remote Console
VMRC
Terminal - hadoop@had...
Terminal - hadoop@hadoop:~/Desktop/Tutorial

Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1771259484242_0038, Tracking URL = http://localhost:8088/proxy/application_1771259484242_0038/
Kill Command = /home/hadoop/hadoop/bin/hadoop job -kill job_1771259484242_0038
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 0
2026-03-22 22:37:08,522 Stage-1 map = 0%, reduce = 0%
2026-03-22 22:37:14,768 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 0.91 sec
MapReduce Total cumulative CPU time: 910 msec
Ended Job = job_1771259484242_0038
MapReduce Jobs Launched:
Job 0: Map: 1 Cumulative CPU: 0.91 sec HDFS Read: 482 HDFS Write: 340 SUCCESS
Total MapReduce CPU Time Spent: 910 msec
OK
101 Laptop Electronics 1200.0 1080.0
102 Phone Electronics 800.0 720.0
103 Headphones Electronics 150.0 135.0
104 Desk Chair Furniture 200.0 180.0
105 Desk Furniture 350.0 315.0
106 Notebook Stationery 5.0 4.5
107 Pen Stationery 2.0 1.8
108 Monitor Electronics 300.0 270.0
109 Mouse Electronics 25.0 22.5
110 Keyboard Electronics 45.0 40.5
Time taken: 14.029 seconds, Fetched: 10 row(s)
hive>
```

The output shows the prices of discounted products which was made possible by use of CTE which creates an intermediate dataset containing the calculated values. It includes columns like the original products table but with an additional column showing discounted price for that product which was created by manipulating data from the original products table.

Query-6: Total Quantity Sold Per Product

Query:

```
SELECT
  p.product_name,
  SUM(s.quantity) AS total_units_sold
FROM sales s
```

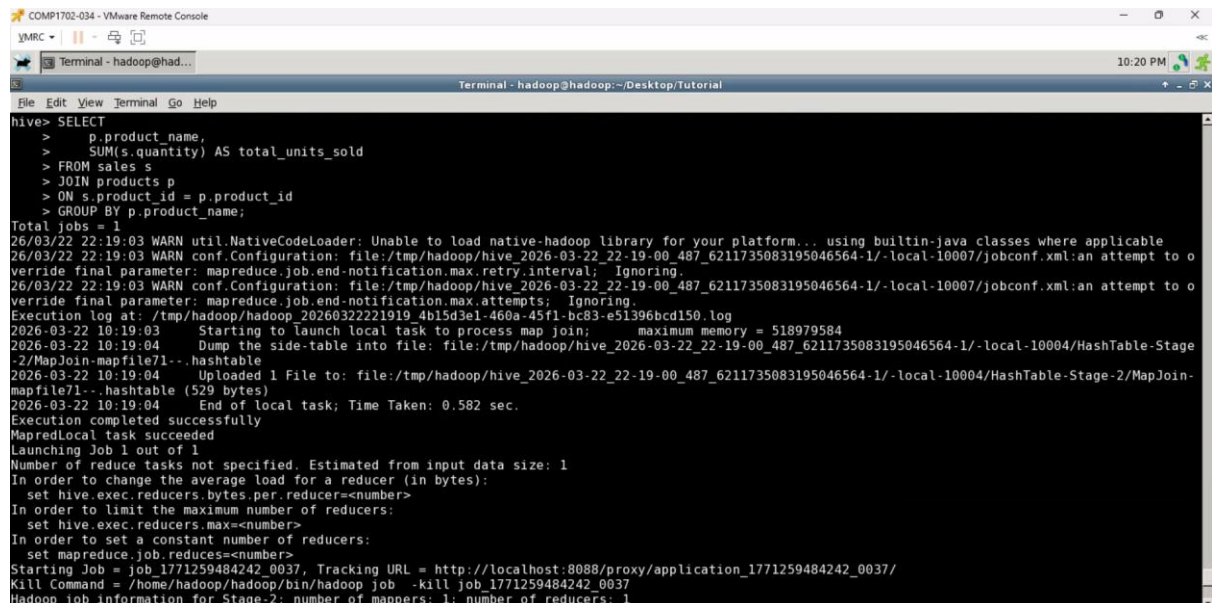
JOIN products p

ON s.product_id = p.product_id

GROUP BY p.product_name;

This is a data analytics query computes the total number of units sold for each product by joining the sales and products tables on the product ID. It uses the SUM aggregation function to calculate total sales quantity and groups the results by product name to provide a summary of units sold per product.

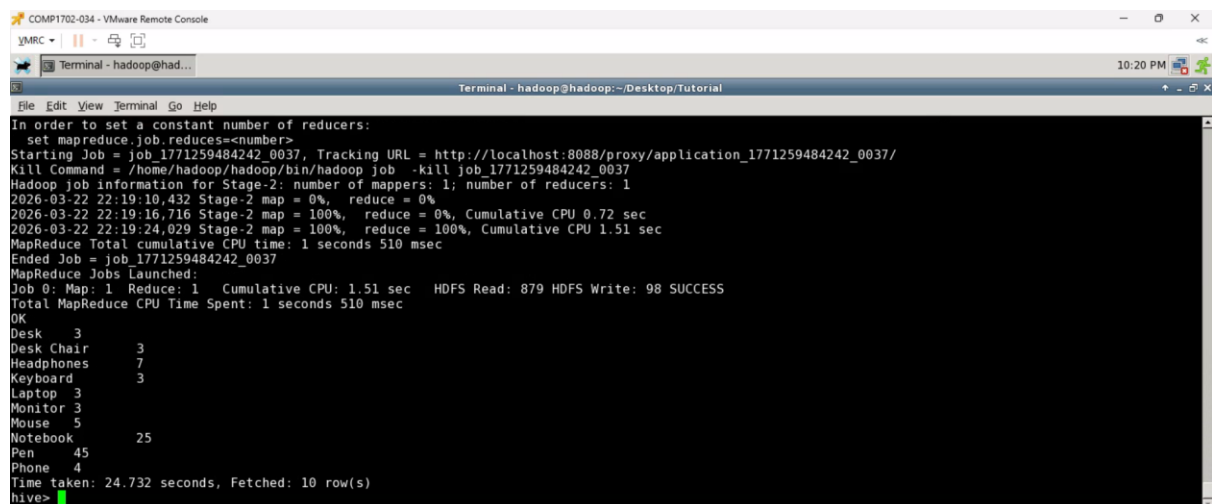
Execution:



```
hive> SELECT
>   p.product_name,
>   SUM(s.quantity) AS total_units_sold
> FROM sales s
> JOIN products p
> ON s.product_id = p.product_id
> GROUP BY p.product_name;

Total jobs = 1
26/03/22 22:19:03 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
26/03/22 22:19:03 WARN conf.Configuration: file:/tmp/hadoop/hive_2026-03-22_22-19-00_487_6211735083195046564-1/-local-10007/jobconf.xml:an attempt to o
verride final parameter: mapreduce.job.end-notification.max.retry.interval; Ignoring.
26/03/22 22:19:03 WARN conf.Configuration: file:/tmp/hadoop/hive_2026-03-22_22-19-00_487_6211735083195046564-1/-local-10007/jobconf.xml:an attempt to o
verride final parameter: mapreduce.job.end-notification.max.attempts; Ignoring.
Execution log at: /tmp/hadoop/hadoop_20260322221919_4b15d3e1-460a-45f1-bc83-e51396bcd150.log
2026-03-22 10:19:03 Starting to launch local task to process map join; maximum memory = 518979584
2026-03-22 10:19:04 Dump the side-table into file: file:/tmp/hadoop/hive_2026-03-22_22-19-00_487_6211735083195046564-1/-local-10004/HashTable-Stage
-2/MapJoin-mappfile71...-hashtable
2026-03-22 10:19:04 Uploaded 1 File to: file:/tmp/hadoop/hive_2026-03-22_22-19-00_487_6211735083195046564-1/-local-10004/HashTable-Stage-2/MapJoin-
mappfile71...-hashtable (529 bytes)
2026-03-22 10:19:04 End of local task; Time Taken: 0.582 sec.
Execution completed successfully
MapredLocal task succeeded
Launching Job 1 out of 1
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1771259484242_0037, Tracking URL = http://localhost:8088/proxy/application_1771259484242_0037/
Kill Command = /home/hadoop/hadoop/bin/hadoop job -kill job_1771259484242_0037
Hadoop job information for Stage-2: number of mappers: 1; number of reducers: 1
```

Output:



```
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1771259484242_0037, Tracking URL = http://localhost:8088/proxy/application_1771259484242_0037/
Kill Command = /home/hadoop/hadoop/bin/hadoop job -kill job_1771259484242_0037
Hadoop job information for Stage-2: number of mappers: 1; number of reducers: 1
2026-03-22 22:19:10,432 Stage-2 map = 0%, reduce = 0%
2026-03-22 22:19:16,716 Stage-2 map = 100%, reduce = 0%, Cumulative CPU 0.72 sec
2026-03-22 22:19:24,029 Stage-2 map = 100%, reduce = 100%, Cumulative CPU 1.51 sec
MapReduce Total cumulative CPU time: 1 seconds 510 msec
Ended Job = job_1771259484242_0037
MapReduce Jobs Launched:
Job 0: Map: 1 Reduce: 1 Cumulative CPU: 1.51 sec HDFS Read: 879 HDFS Write: 98 SUCCESS
Total MapReduce CPU Time Spent: 1 seconds 510 msec
OK
Desk 3
Desk Chair 3
Headphones 7
Keyboard 3
Laptop 3
Monitor 3
Mouse 5
Notebook 25
Pen 45
Phone 4
Time taken: 24.732 seconds, Fetched: 10 row(s)
hive>
```

The output provides sales performance analysis per product by grouping them together. It consists of product name and the total units sold for that product.

Query-7: Find Customers Who Spent More Than £300

Query:

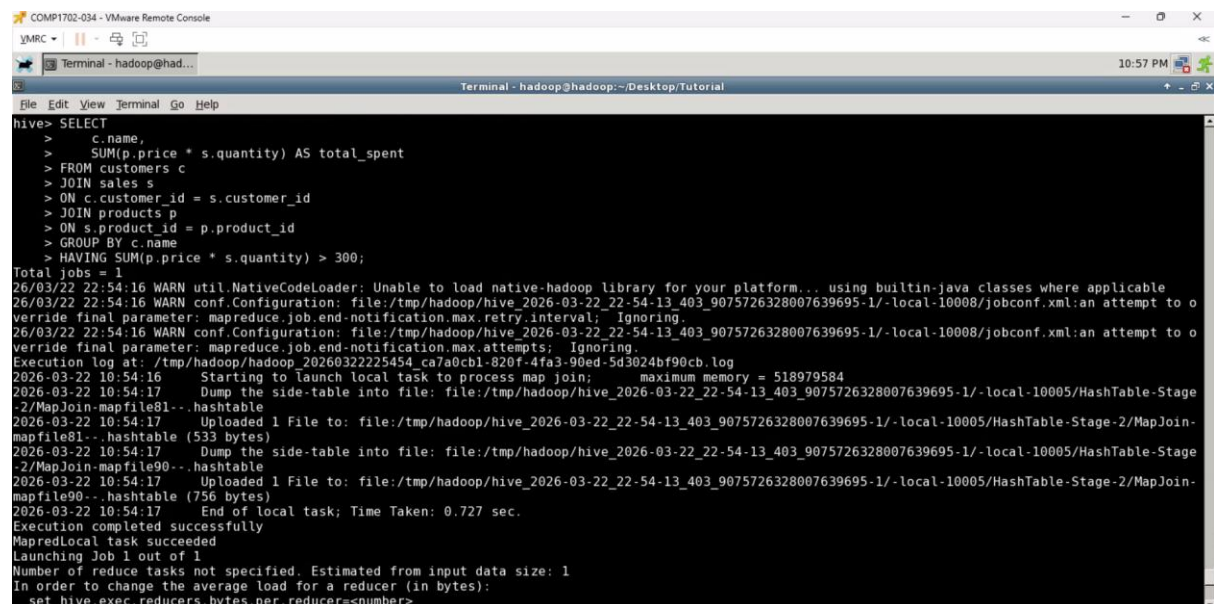
```

SELECT
    c.name,
    SUM(p.price * s.quantity) AS total_spent
FROM customers c
JOIN sales s
ON c.customer_id = s.customer_id
JOIN products p
ON s.product_id = p.product_id
GROUP BY c.name
HAVING SUM(p.price * s.quantity) > 300;

```

This is a data analytics query which analyzes customer spending behavior. It calculates the total amount spent by each customer by joining the customers, sales, and products tables. It aggregates purchase values using the SUM function and groups the results by customer name. A HAVING clause is applied to filter and display only those customers whose total spending exceeds 300.

Execution:



```

hive> SELECT
>     c.name,
>     SUM(p.price * s.quantity) AS total_spent
> FROM customers c
> JOIN sales s
> ON c.customer_id = s.customer_id
> JOIN products p
> ON s.product_id = p.product_id
> GROUP BY c.name
> HAVING SUM(p.price * s.quantity) > 300;

Total jobs = 1
26/03/22 22:54:16 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
26/03/22 22:54:16 WARN conf.Configuration: file:/tmp/hadoop/hive_2026-03-22_22-54-13_403_9075726328007639695-1/-local-10008/jobconf.xml:an attempt to o
verride final parameter: mapreduce.job.end-notification.max.retry.interval; Ignoring.
26/03/22 22:54:16 WARN conf.Configuration: file:/tmp/hadoop/hive_2026-03-22_22-54-13_403_9075726328007639695-1/-local-10008/jobconf.xml:an attempt to o
verride final parameter: mapreduce.job.end-notification.max.attempts; Ignoring.
Execution log at: /tmp/hadoop/hadoop_20260322225454_ca7a0cbl-820f-4fa3-90ed-5d3024bf90cb.log
2026-03-22 10:54:16 Starting to launch local task to process map join; maximum memory = 518979584
2026-03-22 10:54:17 Dump the side-table into file: file:/tmp/hadoop/hive_2026-03-22_22-54-13_403_9075726328007639695-1/-local-10005/HashTable-Stage
-2/MapJoin-mapfile81...hashtable
2026-03-22 10:54:17 Uploaded 1 File to: file:/tmp/hadoop/hive_2026-03-22_22-54-13_403_9075726328007639695-1/-local-10005/HashTable-Stage-2/MapJoin-
mapfile81...hashtable (533 bytes)
2026-03-22 10:54:17 Dump the side-table into file: file:/tmp/hadoop/hive_2026-03-22_22-54-13_403_9075726328007639695-1/-local-10005/HashTable-Stage
-2/MapJoin-mapfile90...hashtable
2026-03-22 10:54:17 Uploaded 1 File to: file:/tmp/hadoop/hive_2026-03-22_22-54-13_403_9075726328007639695-1/-local-10005/HashTable-Stage-2/MapJoin-
mapfile90...hashtable (756 bytes)
2026-03-22 10:54:17 End of local task; Time Taken: 0.727 sec.
Execution completed successfully
MapredLocal task succeeded
Launching Job 1 out of 1
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
set hive.exec.reducers.bytes.per.reducer=<number>

```

Output:

```
COMP1702-034 - VMware Remote Console
VMRC
Terminal - hadoop@hadoop...
Terminal - hadoop@hadoop:~/Desktop/Tutorial

Launching Job 1 out of 1
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes_per_reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1771259484242_0039, Tracking URL = http://localhost:8088/proxy/application_1771259484242_0039/
Kill Command = /home/hadoop/hadoop/bin/hadoop job -kill job_1771259484242_0039
Hadoop job information for Stage-2: number of mappers: 1; number of reducers: 1
2026-03-22 22:54:24,323 Stage-2 map = 0%, reduce = 0%
2026-03-22 22:54:30,587 Stage-2 map = 100%, reduce = 0%, Cumulative CPU 0.92 sec
2026-03-22 22:54:36,785 Stage-2 map = 100%, reduce = 100%, Cumulative CPU 2.07 sec
MapReduce Total cumulative CPU time: 2 seconds 70 msec
Ended Job = job_1771259484242_0039
MapReduce Jobs Launched:
Job 0: Map: 1 Reduce: 1 Cumulative CPU: 2.07 sec HDFS Read: 879 HDFS Write: 133 SUCCESS
Total MapReduce CPU Time Spent: 2 seconds 70 msec
OK
Alice 2000.0
Bob 1900.0
Charlie 650.0
David 550.0
Emma 375.0
Grace 340.0
Hannah 325.0
Jack 1245.0
Karen 800.0
Nick 350.0
Tina 1200.0
Time taken: 24.519 seconds, Fetched: 11 row(s)
hive>
```

The output simply shows which customers(name) spent on buying different products amounting their total spent to be more than 300. This helps to identify who are the possible high-value customers for the business.

Query-8: Find most expensive product in each category

Query:

SELECT

category,

product_name,

price

FROM (

SELECT

category,

product_name,

price,

RANK() OVER (PARTITION BY category ORDER BY price DESC) AS rank_price

FROM products

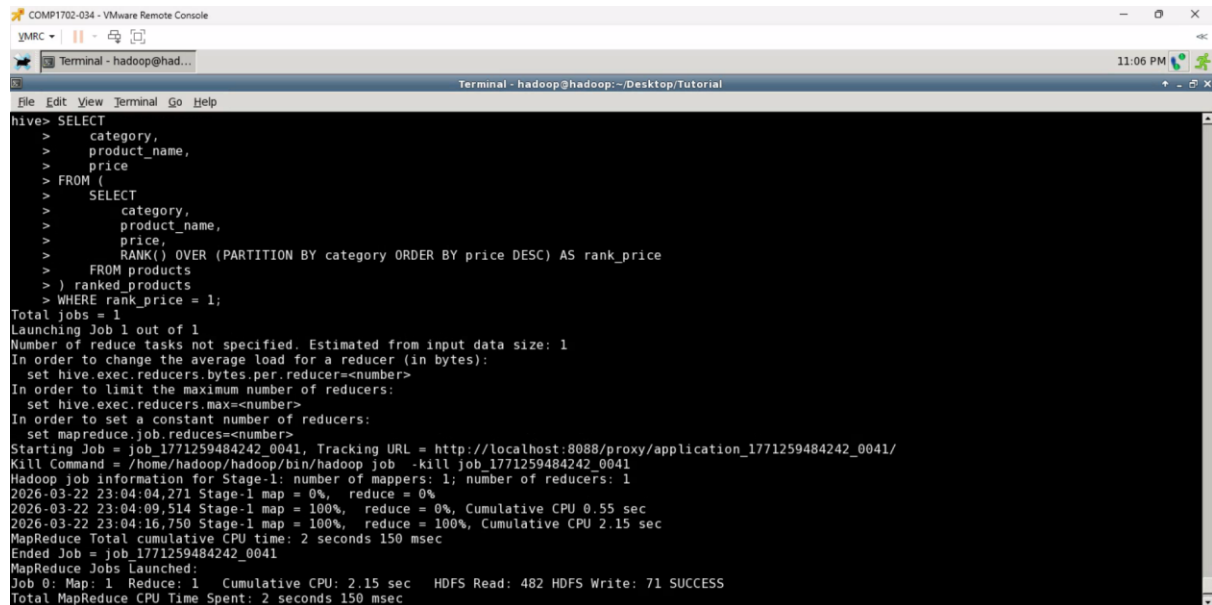
) ranked_products

WHERE rank_price = 1;

This is also a data analytics query which retrieves the most expensive product in each category by using a window function. Window functions allow performing calculations across a set of rows that are related to the current row, without collapsing the result into

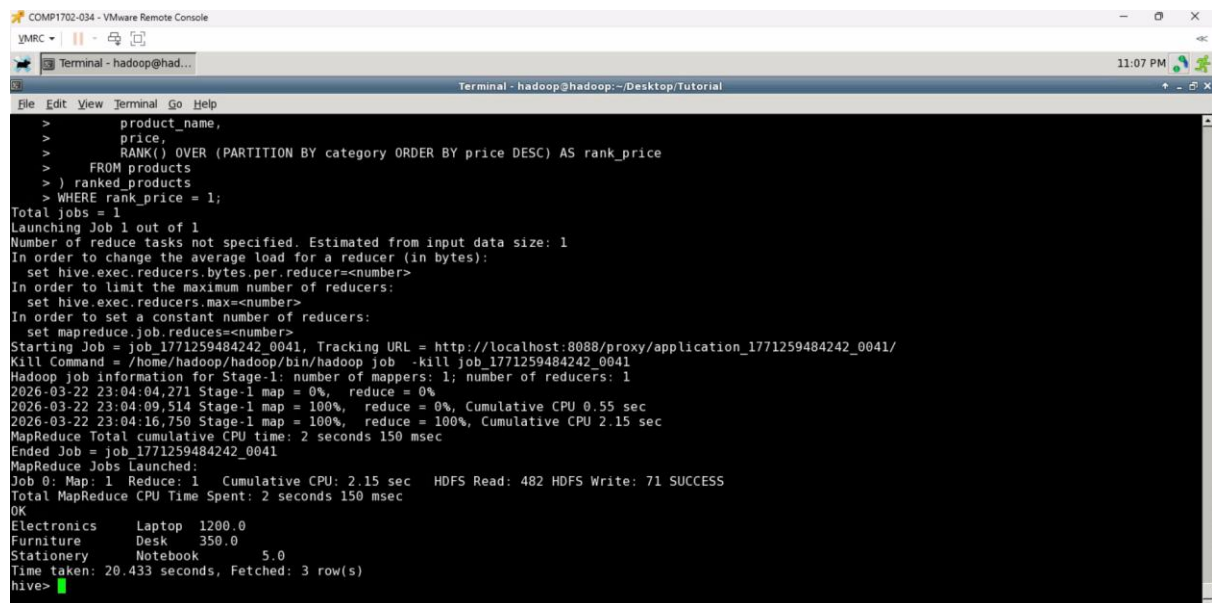
a single value. Here it applies the RANK() function partitioned by category and ordered by price in descending order to rank products within each category. This inner query creates a temporary ranked dataset. Then, the outer query filters the results to include only those with the highest rank (rank = 1), thereby returning the top-priced product(s) for each category.

Execution:



```
hive> SELECT
>   category,
>   product_name,
>   price
> FROM (
>   SELECT
>     category,
>     product_name,
>     price,
>     RANK() OVER (PARTITION BY category ORDER BY price DESC) AS rank_price
>   FROM products
> ) ranked_products
> WHERE rank_price = 1;
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1771259484242_0041, Tracking URL = http://localhost:8088/proxy/application_1771259484242_0041/
Kill Command = /home/hadoop/hadoop/bin/hadoop job -kill job_1771259484242_0041
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-03-22 23:04:04,271 Stage-1 map = 0%, reduce = 0%
2026-03-22 23:04:09,514 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 0.55 sec
2026-03-22 23:04:16,750 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 2.15 sec
MapReduce Total cumulative CPU time: 2 seconds 150 msec
Ended Job = job_1771259484242_0041
MapReduce Jobs Launched:
Job 0: Map: 1 Reduce: 1 Cumulative CPU: 2.15 sec HDFS Read: 482 HDFS Write: 71 SUCCESS
Total MapReduce CPU Time Spent: 2 seconds 150 msec
```

Output:



```
>   product_name,
>   price,
>   RANK() OVER (PARTITION BY category ORDER BY price DESC) AS rank_price
> FROM products
> ) ranked_products
> WHERE rank_price = 1;
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1771259484242_0041, Tracking URL = http://localhost:8088/proxy/application_1771259484242_0041/
Kill Command = /home/hadoop/hadoop/bin/hadoop job -kill job_1771259484242_0041
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-03-22 23:04:04,271 Stage-1 map = 0%, reduce = 0%
2026-03-22 23:04:09,514 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 0.55 sec
2026-03-22 23:04:16,750 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 2.15 sec
MapReduce Total cumulative CPU time: 2 seconds 150 msec
Ended Job = job_1771259484242_0041
MapReduce Jobs Launched:
Job 0: Map: 1 Reduce: 1 Cumulative CPU: 2.15 sec HDFS Read: 482 HDFS Write: 71 SUCCESS
Total MapReduce CPU Time Spent: 2 seconds 150 msec
OK
Electronics      Laptop  1200.0
Furniture        Desk    350.0
Stationery       Notebook  5.0
Time taken: 20.433 seconds, Fetched: 3 row(s)
hive>
```

As the output suggests, it retrieves product category and the product within that category that costs the most. The inner query helps to rank the products within the category and finally outer query selects the top ranked one i.e., the highest priced.

TASK C:

My Student ID number is 001483674 so based on the last digit my task is Task 3.

Pseudo Code:

```
class MAPPER
```

```
    method INITIALIZE
```

```
         $H \leftarrow \text{new ASSOCIATIVEARRAY}$ 
```

```
    method MAP (line_id l, record r)
```

```
        (Student_Id, Gender, Course_Id, Course_Grade, School, Year)  $\leftarrow$  PARSE(r)
```

```
        If Year == "2023" then
```

```
            If  $H\{\text{School}\}$  does not exist then
```

```
                 $H\{\text{School}\} \leftarrow \text{new SET}$ 
```

```
                ADD Student_Id to  $H\{\text{School}\}$ 
```

```
    method CLOSE
```

```
        for all school  $s \in H$  do
```

```
            for all student_id  $st \in H\{s\}$  do
```

```
                EMIT (school  $s$ , student_id  $st$ )
```

```
class REDUCER
```

```
    method REDUCE (school  $s$ , student_ids [id1, id2, ...])
```

```
         $U \leftarrow \text{new SET}$ 
```

```
        for all student_id  $id \in \text{student\_ids}$  do
```

```
            ADD  $id$  to  $U$ 
```

```
        EMIT (school  $s$ , count SIZE( $U$ ))
```

Algorithm Explanation:

The algorithm is an Advanced MapReduce technique which leverages In-Mapper Combiner approach because it provides better optimization and efficiency than combiners as the aggregations occur in-memory in the Mapper itself. The algorithm calculates the number of unique students (accounting for the possibility that one student can appear in multiple records for selecting multiple courses) per school for the year 2023. The provided example data has attributes consisting of: Student_Id, Gender, Course_Id, Course_Grade, School and Year.

Map Stage:

a) **Input key-value pair:** The MAP function of the mapper receives an input as:
(line_id, record r)

- Key: line_id is the unique identifier of the record which is just the line number for the file.
- Value: record contains the full row of the data having values for each attribute.

For example data:

SC001238, male, COMP1702, 65, CMS school, 2019

Into key-value pair:

(1, "SC001238, male, COMP1702, 65, CMS school, 2019")

b) **Output key-value pair:** The output generated by the Mapper is:
(school s, student_id st)

- Key: The School name (e.g., "CMS school").
- Value: A unique Student_Id (e.g., "SCOO1238").

Example:

Input record:

SC001238, male, COMP1702, 65, CMS school, 2019

Mapper output:

(CMS school, SC001238)

Duplicate scenario for a student taking multiple courses:

SC001238, male, COMP1702, 65, CMS school, 2019

SC001238, male, COMP1712, 65, CMS school, 2019

Mapper output becomes:

(CMS school, SC001238)

(CMS school, SC001238)

Computation:

1. The MAP function first parses the input record into its attributes.
2. It immediately applies a filter that ignores any records where the Year is not 2023.
If the record is from 2023, the mapper does not emit data right away.
3. Instead, it adds the Student_Id to a SET of that School which is inside the local associative array H.
4. Since SET do not keep duplicate records, it accounts for the same students who have different courses from being counted twice within a specific Mapper's data chunk by merging them into a single entry.
5. Once all records in that Mapper's data chunk have been processed, the CLOSE method is triggered, which iterates through the associative array H and finally emits a (school, student_id) pair for every unique student found for each school.

Reduce Stage:

- a) **Input Key-Value Pair:** The reducer receives grouped values by key in the format:
(School, [Student_Id1, Student_Id2, Student_Id3,...])
- Key: School name
 - Values: Iterable list of Student IDs associated with that school, collected from multiple mappers across the cluster.

For example:

(CMS School, [SC001483, SC001483, BA147458])

b) **Output Key-Value Pair:** The final output emitted by the Reducer will be in the format: (school s, count).

- Key: School name
- Value: Final integer count of the unique number of students for that school in 2023.

For example:

(CMS School, 2)

Computation:

1. The REDUCE function initializes a new, empty SET called U.
2. Then, the incoming list of Student IDs is iterated, and each Student_Id is added to the set U.
3. Since Hadoop partitions data across different Mappers, there is possibility that same student being processed by two different Mappers. Like maybe their entry in the file has one subject in one cluster and they again have another entry for another course subject ending up at another cluster.
4. So, adding them to the SET U performs a global deduplication, ensuring that even if a Student_Id occurs multiple times, it will exist in the set once.
5. Finally, the Reducer calculates the size of the set i.e., $SIZE(U)$ and emits the final (school, count) pair.

Shuffling Stage:

The shuffle phase occurs between the Map and Reduce stages which happens as:

1. The Mappers emit their (school, student_id) pairs.
2. The MapReduce framework partitions data by key (School).
3. Then it sorts all the intermediate pairs by the Key and groups all Values (student_id) that share the same Key together.
4. The grouped lists are then transferred over the network to specific Reducer nodes which are assigned to handle those specific keys. This makes sure that for data like the example case having school name “CMS School” all go to the same exact Reducer, guaranteeing an accurate final count.

Example:

Mapper1: (CMS School, SC001483)

Mapper1: (CMS School, BA147458)

Mapper2: (CMS School, SC001483)

Mapper2: (BS School, SM100235)

After shuffle and grouping:

CMS School: [SC001483, BA147458, SC001483]

BS School: [SM100235]

Efficiency of the algorithm:

This algorithm is optimal for Big Data processing as given by the coursework scenario because it avoids the naïve approaches that could overwhelm the cluster's network. It does:

1. **Early Filtering:** The condition Year == "2023" immediately makes the mapper filter the records to only the relevant ones. This prevents unnecessary data from entering the MapReduce pipeline.
2. **In-Mapper Deduplication:** Using sets inside the mapper function ensures duplicate student IDs within the same mapper are removed early, which significantly reduces intermediate key-value pairs and shuffle data size.
3. **Parallel Processing:** MapReduce utilizes multiple mappers and reducers by distributing data among them allowing parallel computation and enhance scalability.
4. **Key-Based Partitioning:** Grouping by school ensures that all student IDs for the same school are processed by a single reducer makes aggregation straightforward.

In-Mapper Combiner and How it Optimizes the Algorithm: In-Mapper Combiner was used to make the algorithm efficient because instead of writing to local disk which combiner

does, writing occurs in-memory in the Mapper itself providing control over when and how local aggregation takes place.

In this case, instead of emitting every (School, Student_Id) pair immediately, the mapper stores the student IDs in a set associated with each school.

Inside the mapper:

CMS School: {SC001483, BA147458}

BS School: {SM100235}

In this way, at the end of map task, it emits only unique student IDs per school.

Output:

(CMS School, SC001483)

(CMS School, BA147458)

(BS School, SM100235)

But without it, many duplicate pairs would be transferred across the network affecting the performance. For example:

Without In-Mapper Combiner:

(CMS School, SC001483)

(CMS School, SC001483)

(CMS School, SC001483)

With In-Mapper Combiner:

(CMS School, SC001483)

In this way, only unique IDs from each mapper are emitted.

In summary, the use of In-Mapper reduces intermediate data, decreases network traffic during shuffle and improves the overall MapReduce performance.

TASK D:

D1.

For this task, Data Lake is my preferred choice as it involves handling over 500 petabytes of data from heterogeneous sources like IoT sensors, satellites, drones, social media and market data. Data warehouses follow schema-on-write approach, requiring data to be structured and transformed before ingestion (*What is a Data Warehouse? - Data Warehouse Explained - AWS*, no date). While this makes querying efficient and suitable for analysis, it is ill-suited in this context. Since the PA company receives agricultural data in diverse formats- structured sensor readings, semi-structured JSON from market APIs, unstructured satellite imagery and streaming social media text, forcing them into a rigid schema would result in significant pre-processing overhead and data loss, ruining the core value of raw data (Miloslavskaya and Tolstoy, 2016).

In contrast, a Data Lake utilizes a schema-on-read approach, allowing the PA company to store data in its native format (i.e., Parquet, JSON, images, etc) and applying structure only when the data is queried (Dixon, 2010). It is architecturally ideal for this use case for several reasons. First of all, a volume as big as 500 petabytes makes traditional Data Warehouse cost-ineffective, but distributed storage such as Hadoop HDFS or cloud-native object stores like AWS S3 provides horizontal scalability which data warehouses cannot match. Second, the company's project plans consist of different workloads such as machine learning for predicting yield, real-time stream processing to monitor the weather, and managing financial risks analysis. It stores raw data at granular level which allows performing ad-hoc analysis at any time (Dixon, 2014). Similarly, Data scientists need this type of data to build accurate models. While, data warehouses are optimized for historical reporting and BI, Data Lakes ability to harness more data, from more sources, in less time, and empowering users to collaborate and analyse data in different ways leads to better, faster decision making (*What is a Data Warehouse? - Data Warehouse Explained - AWS*, no date).

Finally, it can be concluded that a Data Lake successfully shows the necessary characteristics to handle the PA company's big data demands while ensuring utmost efficiency.

Table of References

Dixon, J. (2010) 'Pentaho, Hadoop, and Data Lakes', *James Dixon's Blog*, 14 October. Available at: <https://jamesdixon.wordpress.com/2010/10/14/pentaho-hadoop-and-data-lakes/> (Accessed: 22 March 2026).

Dixon, J. (2014) 'Data Lakes Revisited', *James Dixon's Blog*, 25 September. Available at: <https://jamesdixon.wordpress.com/2014/09/25/data-lakes-revisited/> (Accessed: 22 March 2026).

Miloslavskaya, N. and Tolstoy, A. (2016) 'Big Data, Fast Data and Data Lake Concepts', *Procedia Computer Science*, 88, pp. 300–305. Available at: <https://doi.org/10.1016/j.procs.2016.07.439> .

What is a Data Warehouse? - Data Warehouse Explained - AWS (no date) Amazon Web Services, Inc. Available at: <https://aws.amazon.com/what-is/data-warehouse/> (Accessed: 22 March 2026).

D2.

No, I strongly disagree with the choice of MapReduce as it does not align with the company's real-time requirements. MapReduce leverages distributed computing so its architecture is fundamentally incompatible for low-latency demands.

MapReduce operates as a batch processing framework, dividing computation into discrete Map and Reduce phases executed over static datasets stored on HDFS. It bottlenecks because of its disk-heavy nature. As the intermediate results between the phases are written to disk. This constant I/O, combined with significant overhead of task scheduling for every job, results in high latency which makes responding in few seconds unachievable (Dean and Ghemawat, 2008). For time-critical PA services like real-time weather alerts, monitoring livestock or adjusting irrigation, this latency is operationally unacceptable.

Hence, my recommended alternative is Apache Spark. Unlike MapReduce, Spark uses in-memory computation approach i.e., it processes data-in memory (RAM) rather than writing them to disk between processing stages. By doing so, it can speed up computations by up to 100 times than MapReduce (Zaharia *et al.*, 2016). Most importantly, Spark's Structured Streaming enables continuous, low-latency processing of incoming data streams from sensors, drones, and satellites, supporting micro-batch execution with response in seconds, satisfying the company's near real-time requirements. Additionally, Spark uses Directed Acyclic Graph (DAG) to schedule tasks which allows the engine to optimize the entire multi-stage pipelines, reducing redundancy compared to the rigid, linear two-stage execution of MapReduce.

Finally, Spark also provides a unified engine that supports both batch and streaming data. This means that the PA company can use same framework to handle batch processing on the data lake, machine learning via MLlib for yield prediction and streaming to react against sudden weather changes (Zaharia *et al.*, 2016). Within a single cohesive ecosystem, it effectively handles all the PA company's workloads from continuous computing of rolling yield stats, triggering real-time moisture alerts to serving live prediction models.

To conclude, MapReduce should not be in any consideration for real-time needs, instead Apache Spark and its versatile ecosystem is a superior choice in every department for this project's analytical needs.

Table of References

Dean, J. and Ghemawat, S. (2008) 'MapReduce: simplified data processing on large clusters', Communications of the ACM, 51(1), pp. 107–113. Available at: <https://doi.org/10.1145/1327452.1327492>.

Zaharia, M. et al. (2016) 'Apache Spark: a unified engine for big data processing', Communications of the ACM, 59(11), pp. 56–65. Available at: <https://doi.org/10.1145/2934664>.

Declaration of AI Use

I have used AI while undertaking the assignment in the following ways:

- To develop research questions on the topic – YES
- To create an outline of the topic – No
- To explain concepts – YES
- To support my use of language – YES